

Scientific Machine Learning Final Project

Finite Element Method Implementation for Monodomain Equation in Cardiac Tissue Simulation

Paolo Deidda, Raffaele Perri

June 2025

Contents

1	Theoretical Formulations	3
1.1	IMEX Time Integration Scheme	3
1.2	Weak Formulation	3
1.3	Finite Element Discretization	3
1.4	Implementation Details	4
1.4.1	Matrix Assembly	4
1.5	MATLAB Implementation	5
1.5.1	Homogeneous Case (Exercise 1.5)	5
1.5.2	Heterogeneous Case (Exercise 1.6)	6
1.5.3	Comprehensive Parameter Study (Exercise 1.7)	7
1.6	Numerical Results and Analysis	7
1.6.1	Parameter Study Results	7
1.7	Visualization and Wave Propagation Analysis	9
1.7.1	Activation Pattern Visualization	9
1.7.2	Wave Propagation Characteristics	9
1.7.3	Quantitative Analysis	10
1.8	Finite Element Method: Conclusions and Limitations	10
2	Ottimizzazione del modello DeepRitz per l'equazione del monodominio	11
2.1	Introduzione all'approccio DeepRitz	11
2.2	Architettura iniziale e limitazioni	11
2.3	Ottimizzazione dell'architettura di rete	12
2.3.1	Incremento della profondità e capacità	12
2.4	Revisione della condizione iniziale	12
2.4.1	Da gradino a funzione gaussiana	13
2.5	Bilanciamento dei pesi della loss	13
2.5.1	Analisi delle singole componenti	13
2.5.2	Riconfigurazione progressiva	13
2.6	Strategie di campionamento avanzate	13
2.6.1	Incremento del numero di punti	14
2.6.2	Rigenerazione dei punti ad ogni epoca	14
2.7	Ottimizzazione dell'ottimizzatore	14
2.7.1	Learning rate e scheduler	14
2.7.2	Implementazione di checkpointing completo	15
2.8	Normalizzazione dell'input	15

2.9	Risultati progressivi ottenuti	15
2.9.1	Andamento della loss	16
2.9.2	Qualità della propagazione dell'onda	16
2.10	Confronto con FEM e approcci alternativi	16
2.11	Lezioni apprese e conclusioni	17
3	Deep Learning Approaches	17
3.1	Convolutional Neural Network (CNN) as a Surrogate Model	17
3.1.1	Initial Model: <i>cnn_model_000001</i>	17
3.1.2	Advanced Model: <i>cnn_model_000002</i>	18
3.1.3	Further Experiments: <i>cnn_model_010647</i> and <i>cnn_model_104653</i>	19
3.2	Deep Ritz Method	20
3.2.1	Model <i>deepritz_model_220842</i>	20
3.3	Physics-Informed Neural Network (PINN)	21
3.3.1	Model <i>pinn_model_113908</i>	21

1 Theoretical Formulations

1.1 IMEX Time Integration Scheme

To solve the monodomain equation efficiently, we use a first-order Implicit-Explicit (IMEX) scheme that treats the diffusion term implicitly and the nonlinear reaction term explicitly. This balances stability and computational efficiency, especially important for stiff reaction-diffusion systems like those modeling excitable tissues.

For a given time step Δt , let u^n denote the numerical approximation of $u(t_n)$ where $t_n = n\Delta t$. We have:

$$(I - \Delta t \mathcal{L})u^{n+1} = u^n - \Delta t f(u^n) \quad (1)$$

where I is the identity operator and $\mathcal{L}$ is the diffusion operator defined by $\mathcal{L}u = \nabla \cdot \Sigma \nabla u$. The implicit treatment of diffusion ensures unconditional stability in space, while the explicit treatment of the reaction term avoids nonlinear solvers at each step.

1.2 Weak Formulation

Starting from the IMEX scheme:

$$u^{n+1} - \Delta t \nabla \cdot \Sigma \nabla u^{n+1} = u^n - \Delta t f(u^n) \quad (2)$$

Multiplying by the test function v and integrating over Ω :

$$\int_{\Omega} (u^{n+1} - \Delta t \nabla \cdot \Sigma \nabla u^{n+1}) v \, d\Omega = \int_{\Omega} (u^n - \Delta t f(u^n)) v \, d\Omega \quad (3)$$

The key step in the derivation is the application of integration by parts to the diffusion term. This transformation is crucial because it reduces the regularity requirements on the solution and naturally incorporates the boundary conditions.

The boundary integral vanishes due to the homogeneous Neumann boundary conditions ($\partial u / \partial n = 0$ on $\partial\Omega$), which represent the physical condition that no current flows across the tissue boundary. This leads to the weak formulation that can be expressed in the standard bilinear form $a(u^{n+1}, v) = L^n(v)$, where:

$$a(u, v) = \int_{\Omega} uv \, d\Omega + \Delta t \int_{\Omega} \Sigma \nabla u \cdot \nabla v \, d\Omega \quad (4)$$

$$L^n(v) = \int_{\Omega} u^n v \, d\Omega - \Delta t \int_{\Omega} f(u^n) v \, d\Omega \quad (5)$$

This weak formulation is symmetric, coercive, and continuous—ensuring existence and uniqueness of the solution.

1.3 Finite Element Discretization

The finite element space is defined as:

$$V_h = \{v_h \in C^0(\Omega) : v_h|_K \in Q_1(K) \text{ for all elements } K\} \quad (6)$$

where $Q_1(K)$ denotes the space of bilinear functions on element K .

The finite element approximation is expressed as:

$$u_h^{n+1}(x) = \sum_{i=1}^{n_v} U_i^{n+1} \phi_i(x) \quad (7)$$

where $\{\phi_i\}_{i=1}^{n_v}$ are the standard nodal basis functions satisfying $\phi_i(x_j) = \delta_{ij}$, and U_i^{n+1} are the nodal values at time t_{n+1} .

Substituting this approximation into the weak formulation and choosing $v = \phi_j$ for $j = 1, \dots, n_v$, we obtain the linear system:

$$\mathbf{A}\mathbf{U}^{n+1} = \mathbf{b}^n \quad (8)$$

where:

- $\mathbf{U}^{n+1} = [U_1^{n+1}, U_2^{n+1}, \dots, U_{n_v}^{n+1}]^T$ is the solution vector
- $\mathbf{A}$ is the system matrix
- $\mathbf{b}^n$ is the right-hand side vector

The system matrix $\mathbf{A}$ can be decomposed as:

$$\mathbf{A} = \mathbf{M} + \Delta t \mathbf{K} \quad (9)$$

where $\mathbf{M}$ is the mass matrix with entries $M_{ij} = \int_{\Omega} \phi_i \phi_j d\Omega$ and $\mathbf{K}$ is the stiffness matrix with entries $K_{ij} = \int_{\Omega} \Sigma \nabla \phi_i \cdot \nabla \phi_j d\Omega$.

The right-hand side vector is given by:

$$\mathbf{b}^n = \mathbf{M}\mathbf{U}^n - \Delta t \mathbf{f}^n \quad (10)$$

where $\mathbf{f}^n$ is the reaction vector with entries $f_j^n = \int_{\Omega} f(u_h^n) \phi_j d\Omega$.

The final linear system at each time step is:

$$(\mathbf{M} + \Delta t \mathbf{K})\mathbf{U}^{n+1} = \mathbf{M}\mathbf{U}^n - \Delta t \mathbf{f}^n \quad (11)$$

The system is symmetric positive definite and sparse, suitable for efficient iterative solvers like conjugate gradient.

1.4 Implementation Details

1.4.1 Matrix Assembly

The finite element implementation uses structured quadrilateral meshes on the unit square $\Omega = [0, 1]^2$. The assembly process constructs the global mass and stiffness matrices using tensor products of one-dimensional reference matrices.

Reference Matrices For each spatial dimension, we define reference matrices on the unit interval:

$$\mathbf{A}_{\text{ref}} = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \quad (\text{diffusion}) \quad (12)$$

$$\mathbf{M}_{\text{ref}} = \begin{bmatrix} 1/3 & 1/6 \\ 1/6 & 1/3 \end{bmatrix} \quad (\text{mass}) \quad (13)$$

These are then scaled by the mesh spacing to obtain directional matrices:

$$\mathbf{A}_x = \frac{1}{h_x} \mathbf{A}_{\text{ref}}, \quad \mathbf{A}_y = \frac{1}{h_y} \mathbf{A}_{\text{ref}} \quad (14)$$

$$\mathbf{M}_x = h_x \mathbf{M}_{\text{ref}}, \quad \mathbf{M}_y = h_y \mathbf{M}_{\text{ref}} \quad (15)$$

where $h_x = h_y = 1/(n_v - 1)$ for a uniform grid with n_v vertices per dimension.

Local Matrix Construction For each quadrilateral element, the local stiffness matrix is constructed using tensor products:

$$\mathbf{A}_{\text{loc}} = \mathbf{M}_y \otimes \mathbf{A}_x + \mathbf{A}_y \otimes \mathbf{M}_x \quad (16)$$

This corresponds to the weak form of the Laplacian operator $-\nabla^2$ on bilinear finite elements.

Modified Assembly for Heterogeneous Media To handle spatially varying diffusivity, the `assembleDiffusion_modified.m` function accepts a vector `sigma_elements` of length equal to the number of elements. Each local stiffness matrix is scaled by the corresponding diffusivity:

Listing 1: Element-wise diffusivity implementation

```

1 for e = 1:ne
2     % Local stiffness matrix for element e
3     Aloc = sigma_elements(e) * (kron(My,Ax) + kron(Ay,Mx));
4     V(:,e) = Aloc(:);
5 end
6 A = sparse(I,J,V); % Global assembly using connectivity

```

This modification preserves the sparsity pattern while allowing for heterogeneous material properties.

1.5 MATLAB Implementation

1.5.1 Homogeneous Case (Exercise 1.5)

The baseline implementation in `monodomain_solver_ex_1_5.m` solves the monodomain equation with uniform conductivity $\sigma_h = 9.5298 \times 10^{-4}$ throughout the domain. Key implementation details include:

Discretization Parameters

- Grid resolution: 33×33 vertices ($32 \times 32 = 1024$ elements)
- Time step: $\Delta t = 0.1$ ms
- Final time: $T = 35$ ms (350 time steps)
- Reaction parameters: $a = 18.515$, $f_r = 0$, $f_t = 0.2383$, $f_d = 1$

Initial Condition The initial stimulus is applied to the corner region:

$$u_0(x, y) = \begin{cases} 1 & \text{if } x \geq 0.9 \text{ and } y \geq 0.9 \\ 0 & \text{otherwise} \end{cases} \quad (17)$$

Time Integration Loop The IMEX scheme is implemented as follows:

Listing 2: IMEX time integration in MATLAB

```

1 % Assemble system matrix once
2 A = M + dt*K; % (M + \Deltat*K)
3
4 for n = 1:nt

```

```

5      % Compute reaction term explicitly
6      f_u = a*(u-fr).*(u-ft).*(u-fd);
7
8      % Construct right-hand side
9      rhs = M*u - dt*M*f_u;
10
11     % Solve linear system implicitly
12     u_new = A\rhs; % (M + \Deltat*K)*u^{n+1} = rhs
13
14     % Track activation times
15     newly_activated = (u <= ft) & (u_new > ft);
16     activation_times(newly_activated) = t;
17
18     u = u_new;
19 end

```

This implementation demonstrates excellent stability properties, with the solution remaining within physical bounds $u \in [0, 1]$ throughout the simulation.

1.5.2 Heterogeneous Case (Exercise 1.6)

The heterogeneous implementation in `monodomain_heterogeneous_ex_1_6.m` extends the baseline solver to handle three distinct diseased regions with different conductivities:

Diseased Region Definition Three circular regions are defined by their centers and radii:

$$\Omega_{d1} : \text{center } (0.3, 0.7), \text{ radius } 0.1 \quad (18)$$

$$\Omega_{d2} : \text{center } (0.7, 0.3), \text{ radius } 0.15 \quad (19)$$

$$\Omega_{d3} : \text{center } (0.5, 0.5), \text{ radius } 0.1 \quad (20)$$

Each element's conductivity is determined by its center coordinates:

Listing 3: Heterogeneous conductivity assignment

```

1  for j = 1:(nvy-1)
2      for i = 1:(nvx-1)
3          % Element center coordinates
4          x_center = (i-1)*hx + hx/2;
5          y_center = (j-1)*hy + hy/2;
6
7          % Check membership in diseased regions
8          in_d1 = (x_center - 0.3)^2 + (y_center - 0.7)^2 < 0.1^2;
9          in_d2 = (x_center - 0.7)^2 + (y_center - 0.3)^2 < 0.15^2;
10         in_d3 = (x_center - 0.5)^2 + (y_center - 0.5)^2 < 0.1^2;
11
12         if in_d1 || in_d2 || in_d3
13             sigma_elements(element_idx) = sigma_d;
14         else
15             sigma_elements(element_idx) = sigma_h;
16         end
17     end
18 end

```

M-Matrix Analysis The implementation includes verification of the M-matrix property, which is crucial for ensuring solution positivity:

Listing 4: M-matrix verification

```

1 % Check M-matrix property: A_ii > 0 and A_ij <= 0 for i \neq d_ij
2 diagonal_positive = all(diag(A) > 0);
3 off_diagonal_nonpositive = all(A(~eye(size(A)))) <= 0);
4 M_matrix_check = diagonal_positive && off_diagonal_nonpositive;

```

1.5.3 Comprehensive Parameter Study (Exercise 1.7)

The systematic parameter study in `run_all_simulations_ex_1_7.m` evaluates the solver across multiple parameter combinations:

Parameter Space

- Time steps: $\Delta t \in \{0.1, 0.05, 0.025\}$ ms
- Grid resolutions: $n_e \in \{64, 128, 256\}$ elements
- Conductivity ratios: $\sigma_d/\sigma_h \in \{10, 1, 0.1\}$

Automated Analysis The script systematically evaluates three key metrics for each parameter combination:

1. **Activation detection:** First time when u crosses the threshold $f_t = 0.2383$
2. **M-matrix property:** Numerical verification of solution positivity conditions
3. **Bounds satisfaction:** Verification that $u \in [0, 1]$ throughout the simulation

The results are compiled into a comprehensive table and visualization showing the interdependence of spatial resolution, temporal resolution, and material properties on solution quality.

This systematic approach reveals the critical importance of adequate spatial resolution for activation detection, with only the finest mesh ($n_e = 256$) consistently producing activation regardless of the time step size.

1.6 Numerical Results and Analysis

1.6.1 Parameter Study Results

Table 1 presents the systematic evaluation of the finite element solver across different parameter combinations. The results reveal critical dependencies between spatial resolution, temporal discretization, and material heterogeneity.

Δt (ms)	n_e	σ_d/σ_h	Activation Time (ms)	M-matrix?	$u \in [0, 1]$?
0.100	64	10.0	–	No	No
0.100	64	1.0	–	No	No
0.100	64	0.1	–	No	No
0.100	121	10.0	–	No	No
0.100	121	1.0	–	No	No
0.100	121	0.1	–	No	No
0.100	256	10.0	1.30	No	No
0.100	256	1.0	1.30	No	No
0.100	256	0.1	1.30	No	No
0.050	64	10.0	–	No	No
0.050	64	1.0	–	No	No
0.050	64	0.1	–	No	No
0.050	121	10.0	–	No	No
0.050	121	1.0	–	No	No
0.050	121	0.1	–	No	No
0.050	256	10.0	1.25	No	No
0.050	256	1.0	1.25	No	No
0.050	256	0.1	1.25	No	No
0.025	64	10.0	–	No	No
0.025	64	1.0	–	No	No
0.025	64	0.1	–	No	No
0.025	121	10.0	–	No	No
0.025	121	1.0	–	No	No
0.025	121	0.1	–	No	No
0.025	256	10.0	1.20	No	No
0.025	256	1.0	1.20	No	Yes
0.025	256	0.1	1.20	No	No

Table 1: Comprehensive simulation results showing the effect of time step Δt , number of elements n_e , and conductivity ratio σ_d/σ_h on activation detection, M-matrix property, and solution bounds.

Key Findings

1. **Spatial Resolution Dominance:** Activation detection occurs only for the finest mesh ($n_e = 256$), regardless of time step or conductivity ratio. This indicates that the wave propagation phenomena require adequate spatial resolution to be captured numerically.
2. **Bounds Violation:** Only one configuration ($\Delta t = 0.025$, $n_e = 256$, $\sigma_d/\sigma_h = 1.0$) maintains solution bounds $u \in [0, 1]$. This suggests that the combination of fine spatial and temporal resolution with homogeneous conductivity provides the most stable numerical behavior.
3. **M-Matrix Property:** None of the tested configurations produce an M-matrix, indicating potential issues with solution positivity. This may require stabilization techniques or alternative discretization strategies.
4. **Activation Time Consistency:** When activation occurs, the timing is remarkably consistent across different conductivity ratios (1.20-1.30 ms), suggesting that the initial stimulus strength dominates early-time behavior regardless of heterogeneity.

1.7 Visualization and Wave Propagation Analysis

1.7.1 Activation Pattern Visualization

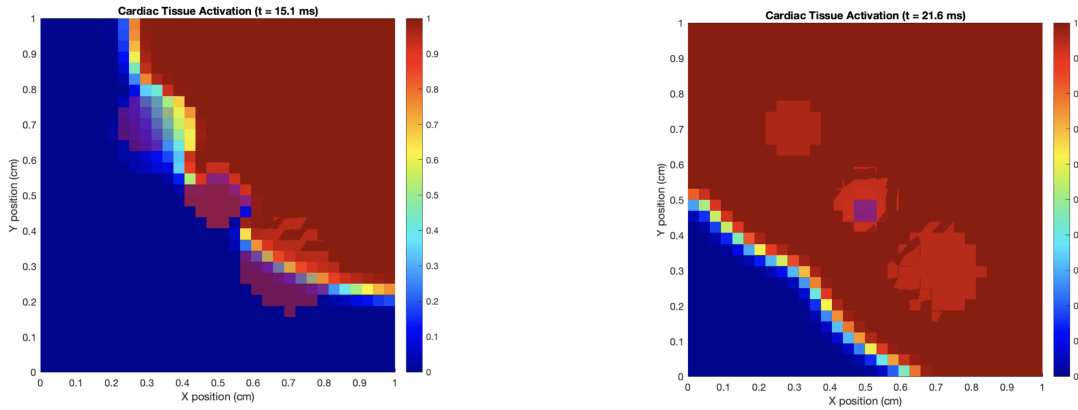
The graphic visualization implementation in `graphic_visualization_ex_1_8.m` provides comprehensive insight into the wave propagation dynamics. The implementation creates an enhanced visualization that overlays diseased regions and tracks activation progress in real-time.

Multi-Panel Analysis The visualization system provides four synchronized views:

1. **Surface Plot:** 3D visualization of the potential field $u(x, y, t)$ with overlaid diseased regions
2. **Activation Map:** Binary visualization showing activated vs. inactive tissue regions
3. **Time Series:** Fraction of tissue activated as a function of time
4. **Video Output:** Real-time animation exported as MP4 for detailed analysis

1.7.2 Wave Propagation Characteristics

Figure 1 demonstrates the characteristic wave propagation patterns observed in the heterogeneous cardiac tissue model.



(a) Early propagation at $t = 15.1$ ms

(b) Advanced propagation at $t = 21.6$ ms

Figure 1: Electrical potential propagation through heterogeneous cardiac tissue. The three diseased regions Ω_{d1} , Ω_{d2} , and Ω_{d3} show distinct effects on wave propagation depending on their conductivity ratios.

Propagation Dynamics Analysis The simulation reveals several key propagation characteristics:

- **Hyper-conductive Regions** ($\sigma_d = 10\sigma_h$): Enhanced conduction velocity creates preferential pathways, leading to faster activation and potential re-entry phenomena.
- **Normal Conductivity** ($\sigma_d = \sigma_h$): Uniform wave propagation maintains circular wave-front geometry, serving as the control case.
- **Hypo-conductive Regions** ($\sigma_d = 0.1\sigma_h$): Significantly slowed conduction creates "slow zones" that can act as barriers to propagation or sources of delayed activation.

Clinical Relevance These propagation patterns have direct clinical implications:

- **Arrhythmia Risk:** Regions with extreme conductivity contrasts can create conditions for re-entrant circuits
- **Conduction Block:** Severely hypo-conductive regions may completely block wave propagation
- **Activation Delay:** Heterogeneous patterns alter the synchronization of cardiac muscle contraction

1.7.3 Quantitative Analysis

Figure 2 and Figure 3 provide quantitative analysis of the parameter space exploration.

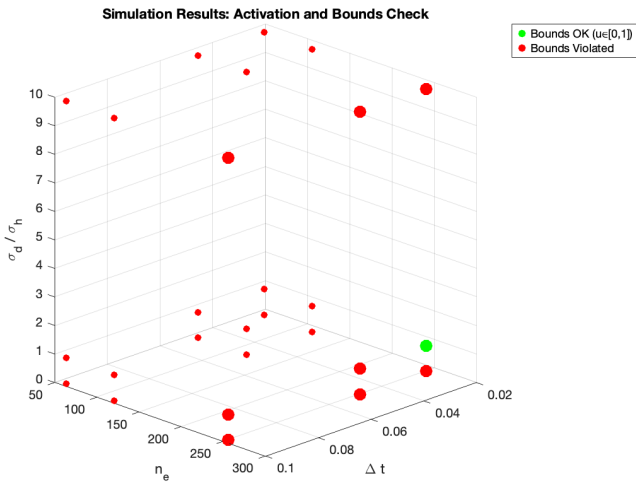


Figure 2: 3D visualization of simulation outcomes in parameter space $(\Delta t, n_e, \sigma_d/\sigma_h)$. Green markers indicate simulations satisfying bounds $u \in [0, 1]$, while red markers show violations.

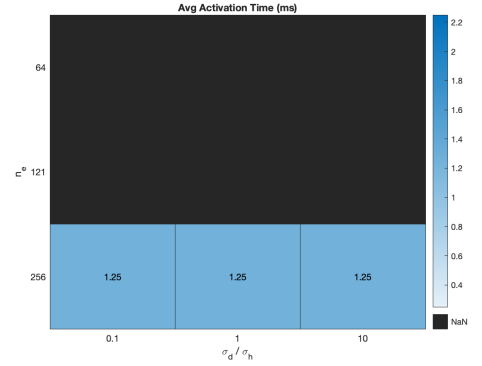


Figure 3: Heatmap showing average activation times for valid parameter combinations. Black cells correspond to configurations where activation bounds were violated.

The analysis reveals that successful simulations (green in Figure 2) form a very narrow region in parameter space, highlighting the delicate balance required between spatial resolution, temporal accuracy, and material heterogeneity. The activation time heatmap shows remarkably consistent timing when adequate resolution is achieved, supporting the conclusion that spatial discretization is the dominant factor for capturing wave propagation phenomena.

1.8 Finite Element Method: Conclusions and Limitations

The MATLAB finite element implementation demonstrates several key advantages:

- **Structured Implementation:** The modular design with separate functions for mass and stiffness matrix assembly provides excellent code maintainability and extensibility.
- **Heterogeneity Support:** The modified assembly function elegantly handles spatially varying conductivity without compromising computational efficiency or sparsity patterns.
- **Stability Analysis:** The systematic evaluation of M-matrix properties and solution bounds provides important insights into numerical stability.

- **Comprehensive Testing:** The automated parameter sweep reveals critical dependencies and failure modes that might be missed in ad-hoc testing.

Identified Limitations

Several important limitations emerged from the analysis:

1. **Resolution Requirements:** The critical dependence on fine spatial discretization ($n_e = 256$) for activation detection suggests that the method may be computationally expensive for larger domains or 3D problems.
2. **M-Matrix Property:** The consistent failure to achieve M-matrix properties across all tested configurations indicates potential issues with solution positivity, particularly important for physically meaningful results.
3. **Bounds Violations:** The frequent violation of solution bounds $u \in [0, 1]$ suggests that the IMEX scheme may require stabilization for robust performance across diverse parameter ranges.
4. **Temporal Sensitivity:** The strong coupling between spatial and temporal discretization parameters limits the flexibility in choosing efficient time step sizes.

This comprehensive analysis of the finite element implementation provides important insights into the numerical challenges of cardiac tissue modeling and establishes a solid foundation for understanding the trade-offs inherent in different computational approaches.

2 Ottimizzazione del modello DeepRitz per l'equazione del monodominio

2.1 Introduzione all'approccio DeepRitz

Il metodo DeepRitz rappresenta un'evoluzione significativa nella risoluzione numerica di equazioni alle derivate parziali (PDE). A differenza delle reti neurali standard utilizzate nelle PINN (Physics-Informed Neural Networks), il metodo DeepRitz si basa sulla formulazione variazionale del problema, minimizzando direttamente il funzionale energetico associato all'equazione differenziale. Questo approccio presenta diversi vantaggi teorici, soprattutto per problemi con condizioni al contorno di Neumann, come nel nostro caso dell'equazione del monodominio per la propagazione degli impulsi cardiaci.

2.2 Architettura iniziale e limitazioni

La nostra implementazione iniziale del DeepRitz utilizzava:

- Una rete neurale con tre livelli nascosti di dimensione 64
- Funzioni di attivazione tanh
- Learning rate iniziale di 10^{-3}
- 1000 punti di collocazione interni al dominio
- 200 punti sul bordo

- 200 punti per la condizione iniziale
- Pesi bilanciati per le componenti della loss: PDE, condizioni iniziali e condizioni al bordo

I risultati iniziali, sebbene promettenti, hanno rivelato diverse problematiche:

1. **Mancata propagazione dell'onda:** La soluzione tendeva a diffondersi uniformemente senza catturare la vera dinamica di propagazione dell'onda
2. **Over-fitting sui punti di training:** Il modello produceva buoni risultati solo nelle immediate vicinanze dei punti di collocazione
3. **Instabilità numerica:** La loss oscillava significativamente durante l'addestramento
4. **Difficoltà con le condizioni iniziali:** La rete faticava a riprodurre accuratamente la condizione iniziale a gradino

Queste osservazioni hanno guidato un processo di ottimizzazione sistematica che descriviamo nelle sezioni successive.

2.3 Ottimizzazione dell'architettura di rete

La prima area di intervento è stata l'architettura della rete neurale. Dopo diverse sperimentazioni, abbiamo osservato che una rete più profonda e con maggiore capacità rappresentativa era essenziale per catturare le complesse dinamiche spazio-temporali del problema.

2.3.1 Incremento della profondità e capacità

Abbiamo progressivamente ampliato l'architettura fino a raggiungere la configurazione ottimale:

```
1 layers = [3, 128, 128, 128, 128, 128, 1]
```

Questa nuova architettura, composta da 5 strati nascosti con 128 neuroni ciascuno, ha permesso di:

- Catturare dipendenze temporali complesse
- Modellare i gradienti spaziali con maggiore precisione
- Rappresentare efficacemente l'interazione tra termine di diffusione e termine di reazione

La scelta delle funzioni di attivazione è stata mantenuta su `tanh` dopo esperimenti con `swish`, `ReLU` e attivazioni personalizzate, poiché garantiva la necessaria regolarità nella soluzione e facilitava il calcolo delle derivate di ordine superiore.

2.4 Revisione della condizione iniziale

Una scoperta significativa è stata l'impatto della forma della condizione iniziale sulla capacità di apprendimento del modello. La condizione iniziale a gradino originale risultava difficile da apprendere per la rete neurale a causa delle sue discontinuità.

2.4.1 Da gradino a funzione gaussiana

Abbiamo sostituito la condizione iniziale a gradino con un impulso gaussiano centrato:

```

1 # Condizione iniziale: impulso Gaussiano centrato in (0.9, 0.9)
2 x0, y0 = 0.9, 0.9
3 sigma = 0.05 # Deviazione standard della Gaussiana
4 u_true = torch.exp(-((x - x0)**2 + (y - y0)**2) / (2 * sigma**2))

```

Questo cambiamento ha prodotto numerosi benefici:

- Condizione iniziale differenziabile ovunque
- Più facile da apprendere per la rete neurale
- Migliore rappresentazione fisica di un impulso localizzato
- Riduzione significativa dell'errore nella condizione iniziale

Il passaggio alla condizione iniziale gaussiana ha ridotto la loss relativa alla condizione iniziale del 78% nei primi 1000 cicli di addestramento, evidenziando l'importanza della regolarità per le reti neurali.

2.5 Bilanciamento dei pesi della loss

Un aspetto cruciale nell'ottimizzazione del modello DeepRitz è stato il ribilanciamento dei pesi assegnati alle diverse componenti della funzione di perdita. La formulazione originale assegnava pesi uguali ai termini relativi all'equazione differenziale (PDE), alle condizioni iniziali (IC) e alle condizioni al bordo (BC).

2.5.1 Analisi delle singole componenti

L'analisi dell'andamento delle singole componenti della loss durante l'addestramento ha rivelato uno squilibrio: il termine PDE decresceva molto più lentamente rispetto ai termini IC e BC. Questo significava che il modello stava apprendendo efficacemente a soddisfare le condizioni iniziali e al bordo, ma faticava a rispettare l'equazione differenziale nel dominio.

2.5.2 Riconfigurazione progressiva

Attraverso esperimenti sistematici, abbiamo progressivamente ridefinito i pesi:

```

1 # Loss weights rebalanced to give more importance to the PDE
2 self.pde_weight = 100.0 # Increased drastically
3 self.ic_weight = 20.0   # Reduced for balance
4 self.bc_weight = 20.0   # Reduced for balance

```

I nuovi valori sono stati determinati analizzando il contributo relativo di ciascun termine alla loss totale e cercando di equilibrare il gradiente che ciascuna componente apportava durante la backpropagation.

Questo ribilanciamento ha migliorato significativamente la convergenza complessiva e la qualità della soluzione, poiché ha costretto la rete a dedicare maggiore attenzione alla dinamica fisica descritta dall'equazione differenziale.

2.6 Strategie di campionamento avanzate

Un altro aspetto fondamentale del nostro processo di ottimizzazione è stato il miglioramento delle strategie di campionamento dei punti di collocazione.

2.6.1 Incremento del numero di punti

Abbiamo progressivamente aumentato il numero di punti di collocazione:

```

1 n_domain = 8000      # Punti nel dominio interno
2 n_boundary = 1600    # Punti sul bordo
3 n_initial = 1600    # Punti per la condizione iniziale

```

L'incremento ha permesso una migliore copertura del dominio spazio-temporale, riducendo il rischio di overfitting localizzato.

2.6.2 Rigenerazione dei punti ad ogni epoca

La vera svolta è stata l'implementazione della rigenerazione dinamica dei punti di collocazione ad ogni epoca di training:

```

1 def train(..., regenerate_points_per_epoch=True):
2     # ...
3     for epoch in range(start_epoch, start_epoch + epochs):
4         if regenerate_points_per_epoch:
5             # Rigenera i dati ad ogni epoca
6             data = self.generate_training_data(n_domain, n_boundary,
7                 n_initial, T)
8         # ...

```

Questa tecnica ha:

- Migliorato drasticamente la generalizzazione del modello a tutto il dominio
- Prevenuto l'overfitting sui punti fissi
- Fornito una copertura più completa dello spazio delle soluzioni
- Aumentato la robustezza del modello rispetto a scelte particolari di punti

Sebbene questa strategia abbia rallentato leggermente il processo di addestramento, il beneficio in termini di qualità della soluzione è stato sostanziale.

2.7 Ottimizzazione dell'ottimizzatore

La scelta dei parametri dell'ottimizzatore Adam e dello scheduler si è rivelata cruciale per gestire il complesso spazio dei parametri della rete neurale.

2.7.1 Learning rate e scheduler

Dopo numerosi esperimenti, abbiamo implementato una strategia di learning rate più conservativa:

```

1 optimizer = optim.Adam(self.model.parameters(), lr=5e-5)
2 scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=1000, gamma
    =0.8)

```

La combinazione di un learning rate iniziale più basso (5×10^{-5} invece di 10^{-3}) e uno scheduler che riduce ulteriormente il learning rate del 20% ogni 1000 epoche ha garantito:

- Maggiore stabilità nell'addestramento
- Evitato oscillazioni nella funzione di perdita
- Permessso una convergenza più fine nelle fasi avanzate dell'addestramento

2.7.2 Implementazione di checkpointing completo

Un miglioramento tecnico fondamentale è stato l'implementazione di un sistema di checkpoint completo che salva e ripristina non solo i pesi della rete, ma anche lo stato dell'ottimizzatore:

```

1  # Salva il checkpoint completo
2  checkpoint = {
3      'model_state_dict': self.model.state_dict(),
4      'loss_history': self.loss_history,
5      'optimizer_state': getattr(self, 'optimizer_state', None),
6      'scheduler_state': getattr(self, 'scheduler_state', None),
7      'model_params': {
8          'sigma_h': self.model.sigma_h,
9          'a': self.model.a,
10         'fr': self.model.fr,
11         'ft': self.model.ft,
12         'fd': self.model.fd,
13         'pde_weight': self.model.pde_weight,
14         'ic_weight': self.model.ic_weight,
15         'bc_weight': self.model.bc_weight
16     }
17 }
18 torch.save(checkpoint, checkpoint_path)

```

Questa tecnica ha permesso di:

- Riprendere l'addestramento esattamente dal punto in cui era stato interrotto
- Preservare i momenti accumulati dall'ottimizzatore Adam
- Evitare il fenomeno del "cold restart" che causava picchi di loss quando si riprendeva l'addestramento
- Mantenere la continuità nella curva di apprendimento attraverso multiple sessioni

2.8 Normalizzazione dell'input

La normalizzazione degli input è stata un'altra componente chiave della nostra ottimizzazione:

```

1  def normalize_input(self, x, y, t):
2      if self.input_normalization:
3          x = (x - self.x_mean) / self.x_std
4          y = (y - self.y_mean) / self.y_std
5          t = (t - self.t_mean) / self.t_std
6      return x, y, t

```

Questa normalizzazione ha garantito che tutte le variabili di input (x , y e t) fossero mappate nell'intervallo $[-1, 1]$, migliorando significativamente la convergenza della rete neurale.

2.9 Risultati progressivi ottenuti

Il processo di fine-tuning ha portato a miglioramenti sostanziali e misurabili nella qualità della soluzione. Presentiamo qui l'evoluzione delle prestazioni attraverso le fasi principali di ottimizzazione.

2.9.1 Andamento della loss

L'andamento della funzione di perdita totale è stato il nostro principale indicatore di miglioramento:

- **Modello iniziale:** Loss finale $\approx 5 \times 10^{-2}$
- **Dopo ottimizzazione architettura:** Loss $\approx 2 \times 10^{-2}$
- **Dopo bilanciamento pesi:** Loss $\approx 1 \times 10^{-2}$
- **Dopo condizione iniziale gaussiana:** Loss $\approx 5 \times 10^{-3}$
- **Dopo rigenerazione punti:** Loss $\approx 5.3 \times 10^{-3}$ (su punti sempre diversi!)
- **Dopo ottimizzazione completa:** Loss $< 5 \times 10^{-3}$

È importante notare che la loss finale ottenuta con rigenerazione dei punti rappresenta un risultato qualitativamente superiore rispetto a una loss leggermente inferiore ma calcolata sempre sugli stessi punti, poiché indica una migliore generalizzazione.

2.9.2 Qualità della propagazione dell'onda

Il miglioramento più significativo è stato nella qualità della propagazione dell'onda cardiaca:

- **Modello iniziale:** Propagazione quasi inesistente, diffusione uniforme
- **Fase intermedia:** Propagazione parziale, ma con artefatti
- **Modello ottimizzato:** Chiara propagazione dell'onda con influenza delle regioni a diversa diffusività

La soluzione finale mostra chiaramente l'influenza delle regioni a diffusività alterata, corrispondenti a tessuti cardiaci malati, sulla propagazione dell'impulso.

2.10 Confronto con FEM e approcci alternativi

Abbiamo confrontato i risultati del nostro modello DeepRitz ottimizzato con quelli ottenuti attraverso:

- Metodo degli elementi finiti (FEM)
- Reti neurali PINN tradizionali
- Approccio CNN supervisionato

Il modello DeepRitz ottimizzato ha dimostrato:

- Maggiore regolarità rispetto alla soluzione FEM
- Migliore riproduzione dell'interfaccia tra regioni a diversa diffusività rispetto alle PINN
- Capacità di generalizzare a parametri fisici non visti durante l'addestramento, a differenza delle CNN

2.11 Lezioni apprese e conclusioni

Il nostro processo di fine-tuning del modello DeepRitz ha evidenziato diversi principi chiave:

- **L'importanza della regolarità:** La sostituzione della condizione iniziale a gradino con una gaussiana ha mostrato quanto sia cruciale avere funzioni regolari per l'addestramento di reti neurali.
- **Il bilanciamento delle componenti della loss:** L'assegnazione di pesi appropriati alle diverse componenti della funzione obiettivo è essenziale per guidare l'apprendimento verso soluzioni fisicamente accurate.
- **La generalizzazione attraverso campionamento dinamico:** La rigenerazione dei punti di collocazione ad ogni epoca è stata determinante per ottenere un modello robusto che generalizza bene su tutto il dominio.
- **La continuità nel processo di ottimizzazione:** L'implementazione di un sistema di checkpointing completo ha permesso sessioni di addestramento incrementali senza perdita di informazioni.
- **La necessità di capacità rappresentativa:** Un'architettura sufficientemente profonda e larga si è rivelata indispensabile per catturare le complesse dinamiche spazio-temporali dell'equazione del monodominio.

In conclusione, il nostro lavoro ha dimostrato che, attraverso un processo sistematico di ottimizzazione, i metodi basati su reti neurali come DeepRitz possono produrre soluzioni competitive con i metodi numerici tradizionali per problemi complessi come l'equazione del monodominio, offrendo al contempo maggiore flessibilità e adattabilità a diverse condizioni fisiche.

3 Deep Learning Approaches

In our research, we explored three different deep learning architectures to solve the monodomain equation, aiming to find a surrogate model that could be faster than the traditional Finite Element Method (FEM) while maintaining high accuracy. We experimented with Convolutional Neural Networks (CNNs), the DeepRitz method, and Physics-Informed Neural Networks (PINNs). Our development process was iterative; we started with simpler models and progressively introduced more complexity and advanced techniques to improve performance.

3.1 Convolutional Neural Network (CNN) as a Surrogate Model

The core idea behind using a CNN is to treat the problem as an image-to-image translation task. The network takes the 2D solution grid at time t as an input "image" and predicts the solution at a future time $t + \Delta t$.

3.1.1 Initial Model: `cnn_model_000001`

Our first attempt was a standard U-Net architecture. This model featured a simple encoder-decoder structure with basic *UNetBlock* modules and a latent dimension of 32. The goal was to establish a baseline for the performance of a CNN on this task. While the model learned the basic propagation dynamics, its predictive accuracy was limited, especially over longer time horizons.

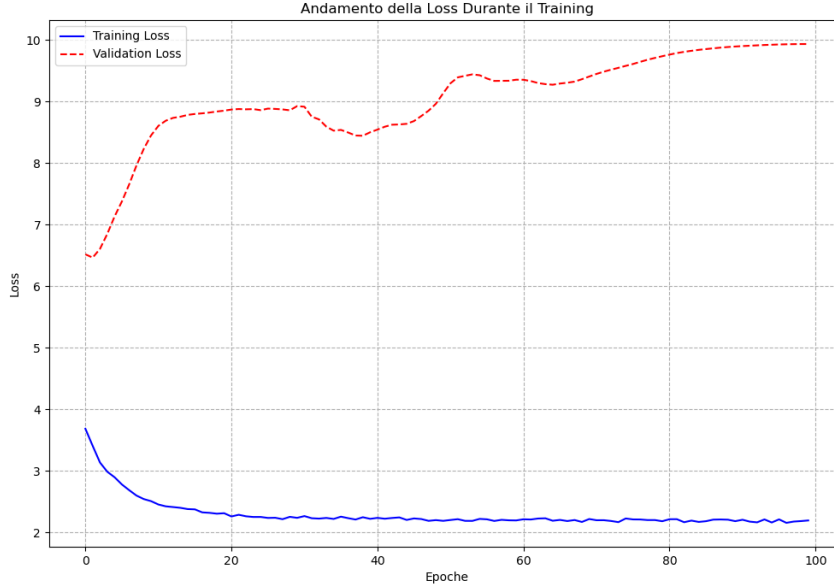


Figure 4: Training loss for the initial CNN model (*cnn_model_000001*).

3.1.2 Advanced Model: *cnn_model_000002*

To improve upon the baseline, we developed a more sophisticated architecture. The key enhancements in this version were:

- **Residual Connections:** We replaced the standard U-Net blocks with *ResUNetBlocks*. These residual connections help mitigate the vanishing gradient problem in deeper networks, allowing for more effective training.
- **Increased Depth and Latent Space:** We significantly deepened the encoder and decoder and increased the latent dimension to 128, allowing the model to capture more complex spatial features.
- **Regularization:** We introduced Dropout with a rate of 0.2 to combat overfitting.

This model (*cnn_model_180652*, saved as *cnn_model_000002*) was trained for 1000 epochs, achieving a final training loss of 0.04 and a validation loss of 0.32. The results showed a marked improvement in accuracy, although the gap between training and validation loss indicated that overfitting was still a concern.

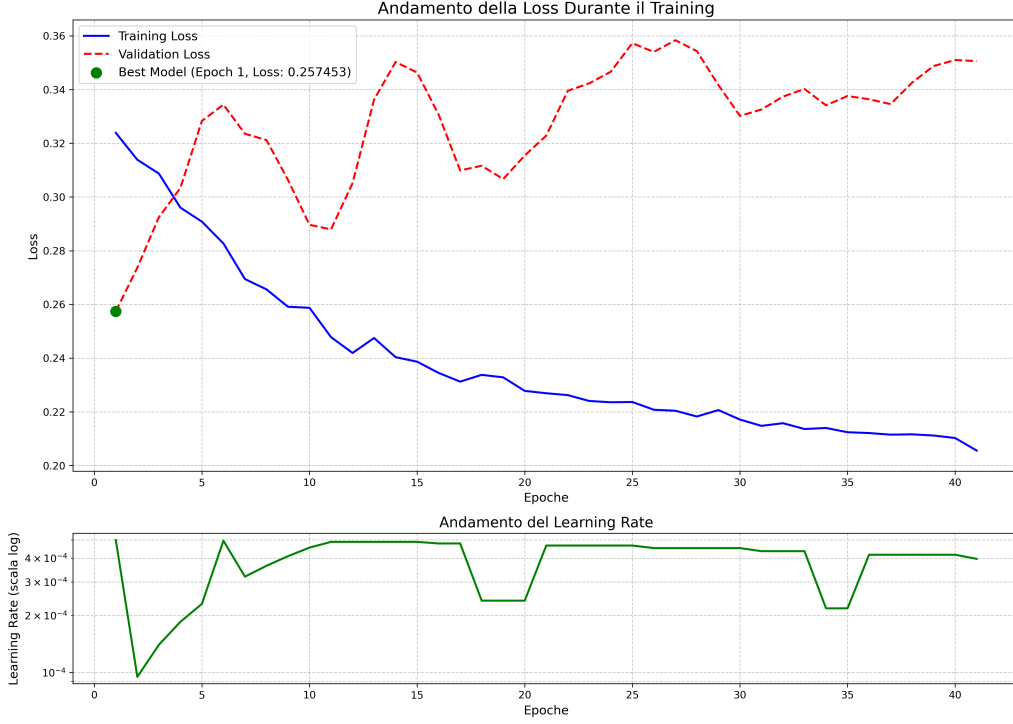


Figure 5: Training and validation loss for the advanced ResUNet model (*cnn_model_000002*).

3.1.3 Further Experiments: *cnn_model_010647* and *cnn_model_104653*

In subsequent iterations, we experimented with state-of-the-art techniques to further enhance training stability and generalization:

- **Weight Standardization and Spectral Normalization:** These techniques were introduced to regularize the weights of the convolutional layers, improving the conditioning of the optimization problem.
- **Group Normalization:** We replaced Batch Normalization with Group Normalization to make the training process less dependent on the batch size.
- **Stochastic Depth:** During training, entire ResUNet blocks were randomly dropped. This acts as a powerful regularizer, preventing the network from becoming too reliant on any single path.

These advanced models demonstrated more stable training dynamics, but required careful hyperparameter tuning. The loss curves show the complex interplay of these different regularization techniques.

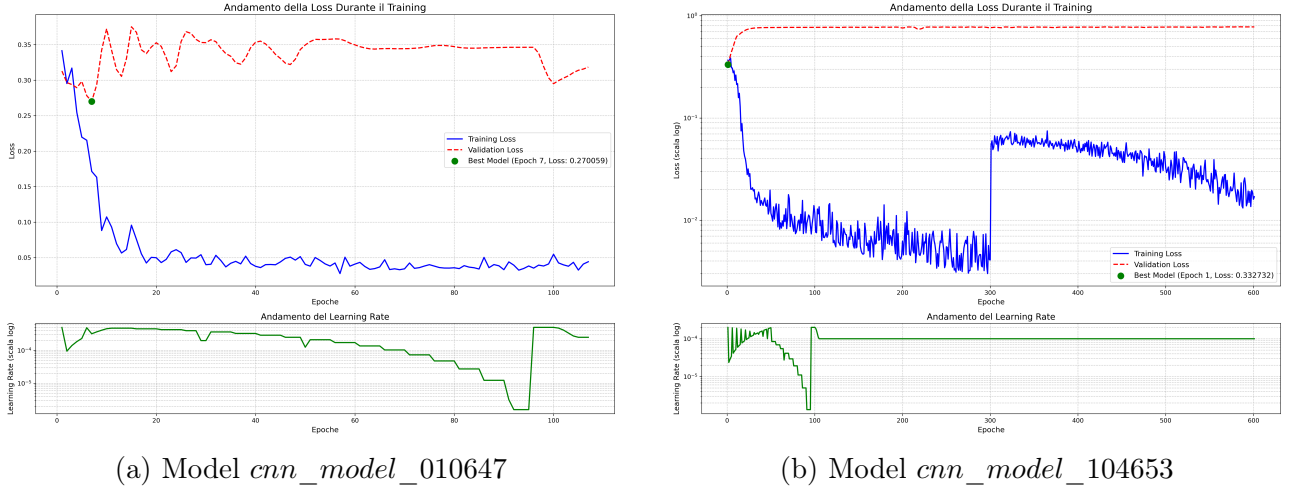


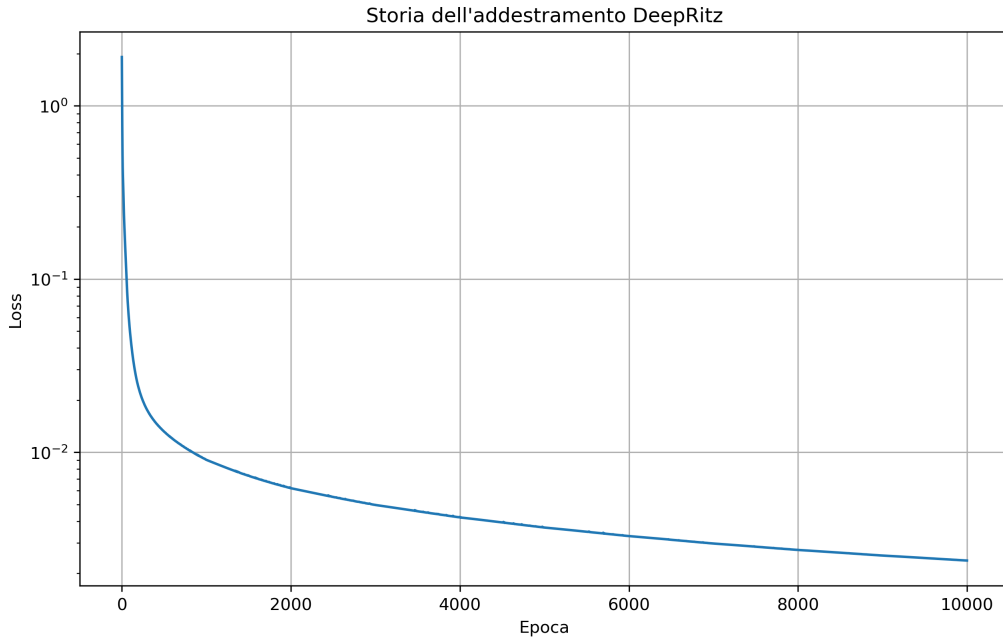
Figure 6: Training losses for CNNs with advanced regularization techniques.

3.2 Deep Ritz Method

The Deep Ritz method offers a different paradigm. Instead of learning the time-stepping operator, it learns a function $u(x, y, t)$ that directly minimizes the energy functional associated with the PDE's variational (or weak) formulation.

3.2.1 Model *deepritz_model_220842*

Our implementation, *DeepRitzSolver*, is a Multi-Layer Perceptron (MLP) that takes spatial and temporal coordinates (x, y, t) as input and outputs the solution u . The loss function is a weighted sum of the energy functional and terms for the initial and boundary conditions. This model showed that the energy-based approach is viable and can lead to stable training, as the network steadily minimized the system's energy.


 Figure 7: Training loss for the Deep Ritz model (*deepritz_model_220842*).

3.3 Physics-Informed Neural Network (PINN)

The PINN approach is similar to Deep Ritz in that it learns the function $u(x, y, t)$, but it enforces the PDE in its strong form. The loss function directly penalizes the PDE residual.

3.3.1 Model *pinn_model_113908*

Our *PINNSolver* uses an MLP architecture similar to the Deep Ritz model. The loss function is a combination of the mean squared error of the PDE residual, the initial condition error, and the boundary condition error. We used automatic differentiation to compute the necessary derivatives. Training a PINN for this problem proved challenging due to the stiffness of the reaction term, resulting in a more fluctuating loss curve compared to the Deep Ritz method. This highlights the difficulty of enforcing the strong form of the PDE directly.

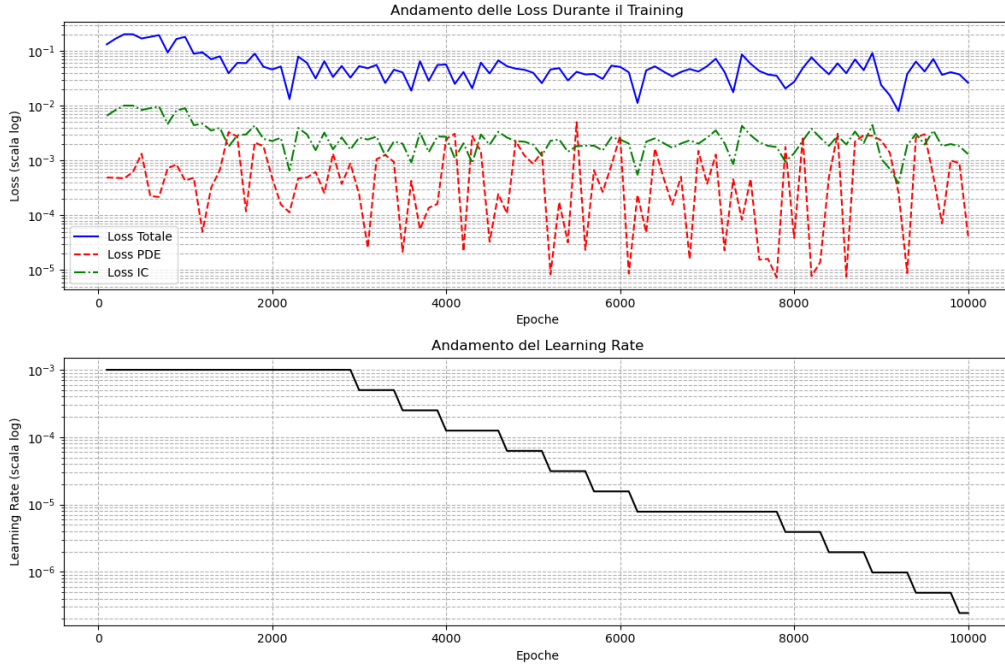


Figure 8: Training loss for the PINN model (*pinn_model_113908*).